Informe final — Calidad de Software aplicada al TFG: NutriScan

Trabajo Integrador · Calidad de Software · Fecha de presentación: 26 de octubre de 2026

Nivel declarado: 3 — prototipo avanzado

Integrantes: *(completar con nombres)* · **Repositorio:**
https://github.com/baujuncos/NutriScan_ProyectoIntegrador (rama `tp-calidad`)

> Todas las cifras de este informe provienen de ejecuciones del propio proyecto (comando y archivo de evidencia indicados en cada caso). Lo que todavía depende de una acción externa al repositorio se marca como **pendiente**; no se estima ni se inventa ningún resultado.

---

1. Resumen

NutriScan es una aplicación de nutrición deportiva (Next.js 16, Supabase, Gemini, Open Food Facts) usada en un estudio científico. Se aplicó un proceso de calidad completo y automatizado: plan de pruebas con riesgos, 1884 pruebas automáticas + 15 E2E contra servicios reales, un pipeline de integración continua con *quality gate*, una verificación automática después de cada despliegue (smoke, E2E, carga) y un registro de 19 defectos.

Resultados principales:

- La cobertura real del código era **70,34 %**, no el 86,11 % que se reportaba (la herramienta no contaba los archivos sin tests). Hoy es **84,82 %** de sentencias y **81,21 %** de decisiones, medida sobre todo `src/`.
- Se encontraron **11 defectos nuevos**, entre ellos un endpoint abierto que filtraba los emails de todos los usuarios (DEF-002) y, gracias a la prueba contra la integración real, una **política de base de datos que permite leer perfiles sin iniciar sesión** (DEF-009, abierto: requiere ejecutar una sentencia SQL en Supabase).
- ESLint pasó de **35 errores a 0**; las vulnerabilidades críticas de producción, de **1 a 0**.
- Stress y esfuerzo sostenido ejecutados con umbrales: una instancia soporta 100 usuarios simultáneos con p95 de 463 ms y sin errores; con 300 usuarios la latencia p95 llega a 2,8 s (sin errores) y se incumple el umbral.

2. Nivel declarado y alcance

Se declara el **Nivel 3 (prototipo avanzado)**, porque la tesis está casi terminada y tiene tres integraciones externas funcionando. Alcance: toda la aplicación y sus integraciones con Supabase (Auth, base con RLS, Storage), Gemini (chat y foto) y Open Food Facts. Fuera de alcance: pruebas con usuarios, dispositivos físicos reales y las bibliotecas de terceros en sí.

3. Metas del nivel 3: cumplidas y no cumplidas

Estado: ✅ cumplida y verificada · 🟡 implementada y verificada **localmente**, falta su primera ejecución en GitHub/Vercel · ⏳ depende de una acción del equipo.

Aspecto	Exigido (nivel 3)	Estado	Evidencia / observación
Pipeline	Compilación, pruebas, análisis estático, cobertura, quality gate y reporte automático	🟡	`.github/workflows/ci.yml`; todos sus pasos ejecutados en local con `npm run qa` (`docs/evidencia/quality-gate.md`). Falta la ejecución en GitHub Actions (requiere `push`)
Quality gate	Bloquea ante bugs críticos, duplicación > 5 % y cobertura de código nuevo < 80 %	✅	`scripts/qa/quality-gate.mjs` + 21 pruebas del propio gate. Aprobado: 0 errores, 2,62 % de duplicación, 98,57 % de código nuevo
Cobertura de código nuevo	80 %	✅	98,57 % (69/70 líneas)
Plan de pruebas y riesgos	Versionado, criterios de entrada/salida, riesgos P×I, trazabilidad	✅	`docs/plan-de-pruebas.md` (14 riesgos priorizados, trazabilidad requisito → caso → defecto)
Defectos	Con pasos, esperado y obtenido, vinculados a su caso, en Jira u otra herramienta	✅ / ⏳	`docs/defectos.md` (19 defectos) y `docs/defectos.csv` listo para importar a Jira. La carga en Jira la hace el equipo
Herramienta nueva	Proceso de 5 pasos	✅	k6: `docs/herramienta-k6.md` (requisitos, mercado, PoC, matriz, despliegue incremental). Etapas 4–5 del despliegue (en el pipeline) 🟡
Cobertura mínima del código existente	80 %	✅	84,82 % de sentencias; 81,21 % de decisiones
Smoke test	Ejecutado por el pipeline tras el despliegue	🟡	`scripts/smoke.mjs` y `deploy-verify.yml`. Ejecutado en local: 8/8 (`docs/evidencia/smoke-local.md`)
Integración externa	Flujo E2E automatizado en cada despliegue	🟡 / ⏳	`testing/e2e/` (15 pruebas). Ejecutado en local contra los servicios reales: 7 pasan, 1 falla (detectó DEF-009), 7 se saltean por falta del usuario de prueba (⏳)
Stress y esfuerzo sostenido	Ejecutado con resultados y umbrales	✅ (local) 🟡 (pipeline)	`docs/pruebas-de-carga.md`, `docs/evidencia/carga/`. Falta la corrida contra Vercel desde el pipeline
Entregable: ejecución en verde y ejecución fallida que detectó un problema real	Pipeline con ambas	⏳	Los commits están preparados (`c220b12` rojo → `da001fb` verde); hay que hacer `push` para que queden en el historial de Actions
Entregable: demostración con despliegue nuevo	Integraciones, pruebas y cargas	⏳	`docs/guion-demo.md`

No cumplido / abierto, dicho con claridad: (1) las ejecuciones en GitHub Actions y contra Vercel todavía no existen; (2) DEF-009 y DEF-004 siguen abiertos; (3) el E2E del flujo completo (login → chat con IA →

guardado) no se pudo ejecutar porque falta crear el usuario de prueba.

4. Evolución de las métricas

Métrica	Antes	Después	Fuente
Cobertura de sentencias, todo `src/`	70,34 %	**84,82 %** (+14,48 pp)	`vitest --coverage`
Cobertura de decisiones	68,16 %	**81,21 %** (+13,05 pp)	idem
Cobertura que reportaba el repo	86,11 % (inflada)	—	DEF-006
Errores de ESLint	35	**0**	`eslint . -f json`
Warnings de ESLint	99 (85 de un archivo generado)	18	idem
Vulnerabilidades críticas (prod)	1	**0**	`npm audit --omit=dev`
Vulnerabilidades totales (prod)	16	8	idem
Duplicación	2,67 %	2,62 %	`jscpd`
Pruebas automáticas	1416	1884 (+15 E2E)	`vitest list`
Rutas de API sin ninguna cobertura	7	0	`coverage-summary.json`

Métrica de proceso: cobertura del código nuevo por cambio = **98,57 %**. Métrica de proyecto: densidad de defectos = **0,74 por KLOC** (19 defectos / 25,6 KLOC). Detalle e interpretación en `docs/metricas.md`.

5. Plan de pruebas

(`docs/plan-de-pruebas.md`.) Contiene criterios de entrada y de salida medibles, la matriz de las 8 características de ISO/IEC 25010 con evidencia, riesgos por probabilidad × impacto, prácticas de QA separadas de las de QC, plantillas, objetivos medibles (usuario, equipo, organización), plan de contingencia, los siete principios aplicados y la trazabilidad.

Top 3 de ISO 25010: (1) **Seguridad** — datos de salud y estudio a ciegas; es donde están los defectos más graves; (2) **Adecuación funcional** — el valor del producto son números correctos; (3) **Fiabilidad** — depende de un tercero (Gemini) que puede fallar.

Riesgos más altos: R-01 caída de Gemini (16), R-12 respuesta incorrecta de la IA (16), R-03 fuga de datos por RLS (15, materializado en DEF-009).

6. Pruebas

Pirámide: 933 pruebas unitarias, 603 de componentes, 331 de API/acciones, 17 de integración, 15 E2E y las de carga. Casos documentados con **clases de equivalencia, valores límite y tablas de decisión** en `docs/casos-de-prueba.md`, con cobertura de sentencia y de decisión por módulo. Se aplicó Arrange–Act–Assert y el criterio de aceptación se fijó antes de ejecutar.

Ejemplos de módulos críticos (sentencias / decisiones): cálculos nutricionales 100 % / 100 %; contraseña 100 % / 100 %; recordatorios 100 % / 91,7 %; borrado por email 100 % / 86,4 %; middleware de sesión 100 % / 98 %.

7. Integraciones externas

(`docs/integracion-externa.md` .) Contrato con Supabase, Gemini y Open Food Facts; **dobles de prueba** para el CI rápido (segundos, sin red) y **E2E contra los servicios reales** en cada despliegue. Política de Gemini: 15 s por intento, un reintento, modelo de respaldo, sin reintento ante 429. Resultado más importante: el E2E contra la base real detectó que `profiles` era legible sin sesión (DEF-009), algo imposible de ver con mocks.

8. Pipeline de CI/CD

`ci.yml` (cada push/PR): checkout completo → `npm ci` → `next build` → `tsc` → pruebas con cobertura → ESLint + jscpd + `npm audit` → **quality gate** → resumen y artefactos. `deploy-verify.yml` (cada despliegue de Vercel, evento `deployment_status` , o manual): **smoke → E2E real → k6 (stress y sostenido)**, con informes como artefactos.

Quality gate (ejecución del 23/10/2026, commit `fac8117`):

Chequeo	Valor	Umbral	Resultado
Errores de ESLint	0	0	✅
Duplicación	2,62 %	≤ 5 %	✅
Cobertura global	84,82 %	≥ 80 %	✅
Cobertura de código nuevo	98,57 %	≥ 80 %	✅
Vulnerabilidades críticas (aviso)	0	0	✅

9. Pruebas de stress y esfuerzo sostenido

(`docs/pruebas-de-carga.md` , herramienta **k6 v2.3.0**.) Ejecutadas contra la app compilada en producción en una notebook (8 hilos), k6 y servidor en la misma máquina — por lo que **miden una instancia**, no la capacidad de Vercel.

Prueba	Carga	Solicitudes	p95	p99	Error	Umbrales
Stress 1	hasta 100 usuarios, 4 min 15 s	16 771	463 ms	797 ms	0 %	✅ cumplidos
Stress 2	hasta 300 usuarios, 3 min 50 s	23 086	2848 ms	3122 ms	0 %	❌ p95 global y `/login`
Esfuerzo sostenido	20 usuarios, 10 min	18 107	27 ms	51 ms	0 %	✅ cumplidos, sin degradación (×0,35)

El límite de una instancia está entre 100 y 300 usuarios simultáneos; el cuello de botella es el renderizado dinámico de `/login` . Sin errores en ningún escenario: la degradación es gradual (cola), no una caída.

10. Defectos

(`docs/defectos.md` .) 19 registrados: 3 críticos, 5 altos, 9 medios, 2 bajos; **17 corregidos y 2 abiertos**. Todos tienen caso de prueba; los corregidos, una prueba de regresión.

Contraste rojo/verde (DEF-002). El test, escrito primero, falló contra el código original con `expected 200 to be 401` y mostró que la respuesta incluía los emails (10 de 11 pruebas en rojo, commit `c220b12`). Tras agregar la autorización por `CRON_SECRET` (commit `da001fb`) pasan 23 de 23. Evidencia: `docs/evidencia/def002-rojo.txt` y `def002-verde.txt` .

ID	Defecto	Sev.	Estado
DEF-002	`/api/send-reminders` sin autenticación, filtra emails	Crítica	Corregido
DEF-009	`profiles` legible sin sesión en la base real	Crítica	**Abierto** (SQL listo)
DEF-005	Intentos ilimitados del código de investigador	Alta	Corregido
DEF-008	Hidratación acepta cualquier valor	Alta	Corregido
DEF-003	Borrado por email no encuentra usuarios tras el puesto 50	Media	Corregido
DEF-004	Harris-Benedict con constantes de 1919 documentadas como "revisadas 1984"	Media	**Abierto** (decisión del equipo)
DEF-006	Cobertura inflada (86 % vs 70 % real)	Media	Corregido
DEF-001	Test intermitente por timeout	Media	Corregido
DEF-007, DEF-019	Inconsistencias de validación en login y fecha de nacimiento	Baja	Corregidos
DEF-010 … 018	Históricos (escalada de rol, buscador, Open Food Facts, estudio a ciegas…)	varias	Corregidos

11. Herramienta incorporada: k6

Proceso de cinco pasos (`docs/herramienta-k6.md`): **requisitos** (umbrales como código, CI, JavaScript), **mercado** (k6, Artillery, JMeter, Locust, autocannon), **prueba de concepto** (binario verificado por SHA-256, guion mínimo, escenario real), **matriz de decisión** (k6 4,75; Artillery 3,95; JMeter y Locust 3,15; autocannon 3,10) y **despliegue incremental** (local → scripts en el repo → corridas con umbrales → pipeline manual → pipeline automático).

12. Análisis crítico

- **Lo que se hizo bien.** Medir la cobertura sobre todo el código evitó una falsa tranquilidad; escribir primero el test rojo dio evidencia de que cada prueba detecta de verdad el defecto; el E2E contra el servicio real encontró el hallazgo más grave.
- **Qué no se logró.** No hay todavía ejecuciones en GitHub Actions ni contra Vercel; faltó el usuario de prueba para el flujo E2E completo; las pantallas de servidor (`home` , `perfil` , `elegir-uso` , `encuesta-psicologica`) siguen sin pruebas; hay 46 funciones de complejidad > 15, la mayor (124) en la exportación a Excel.
- **Limitaciones de las mediciones.** La carga se midió sobre una sola instancia y compartiendo CPU con el generador; no incluye el flujo con Gemini (cuota y costo de terceros). La cobertura mide ejecución, no calidad de las aserciones.

- **Decisiones discutibles.** Bajar a *warning* 13 reglas del React Compiler para que el gate bloquee solo bugs reales (queda como deuda visible); no cambiar la fórmula de Harris-Benedict sin acuerdo del equipo; el límite de intentos del código de investigador es por instancia (aproximado).
- **Riesgos remanentes.** R-01 (Gemini), R-05 (fórmula), R-07 (consulta de cumplimiento sin paginar), R-11 (dependencias con cambios mayores pendientes).

13. Conclusiones

1. La calidad de NutriScan se sostiene hoy en **datos medidos y repetibles**: un comando (`npm run qa`) produce las cifras de este informe y el pipeline las vuelve a producir en cada cambio.
2. Los defectos más graves estaban donde no había pruebas (API y acciones del servidor) y donde los mocks no alcanzan (la base real). Por eso la combinación de pruebas unitarias rápidas con pruebas E2E contra servicios reales es lo más valioso del proceso.
3. Para cerrar el trabajo hacen falta cuatro acciones del equipo, todas documentadas: ejecutar la sentencia SQL de DEF-009, cargar los secrets y `CRON_SECRET` (`docs/SETUP_SECRETS.md`), hacer `push` y mergear para obtener las ejecuciones en GitHub, y decidir DEF-004.

Anexo A — Comandos para reproducir

```
npm ci
npm run qa                                  # lint + tipos + cobertura + análisis + quality gate
npm test                                    # 1884 pruebas
BASE_URL=<url> npm run smoke                # smoke
npm run test:e2e                            # E2E real (necesita .env.local)
BASE_URL=<url> npm run load:stress          # k6
BASE_URL=<url> npm run load:soak            # k6
```

Anexo B — Mapa de evidencia

Evidencia	Ubicación
Rojo/verde de cada defecto	`docs/evidencia/defNNN-rojo.txt`, `defNNN-verde.txt`
Quality gate	`docs/evidencia/quality-gate.md`
Cobertura (resumen y por módulo)	`docs/evidencia/coverage-summary.json`, `cobertura-modulos.md`
Smoke y E2E locales	`docs/evidencia/smoke-local.md`, `e2e-local.txt`
Resultados de k6	`docs/evidencia/carga/`